

Disclaimer

Environment: This notebook has been tested with:

- Julia v1.12.x
- JuMP.jl v1.30.x
- HiGHS v1.24.1

Author: Xavier Gandibleux, Nantes (France)

Case 1: Single-Objective Optimization

Given the 01 Unidimensional Knapsack Problem (01UKP) formulated by

$$z = \max \{ px \mid wx \leq c, x \in \{0, 1\}^n \}$$

and the numerical instance corresponding to

$$n = 5$$

$$p = (5, 3, 2, 7, 4)$$

$$w = (2, 8, 4, 2, 5)$$

$$c = 10$$

perform the following progressive tasks:

1. implement an explicit formulation of the 01UKP with JuMP and display the model.
2. compute the optimal solution using HiGHS as MIP solver and display the solution.
3. compute the linear relaxation using HiGHS as LP solver and display the solution.
4. add an constraint and examine the optimal solution from different perspectives.

Solutions:

1. implement and display an explicit formulation of the 01UKP with JuMP.

indications: an explicit formulation is a description in extension of a mathematical model (composed of variable(s), function(s), constraint(s)), and data.

Hints: the formulation to implement in JuMP is the following:

$$\begin{aligned} \max \quad & 5x_1 + 3x_2 + 2x_3 + 7x_4 + 4x_5 \\ \text{Subject to} \quad & 2x_1 + 8x_2 + 4x_3 + 2x_4 + 5x_5 \leq 10 \\ & x_1, x_2, x_3, x_4, x_5 \in \{0, 1\} \end{aligned}$$

declare the package(s) to use:

In []:

setup the formulation:

In []:

display the model:

Hints: see `print()` in the Julia documentation

In []:

2. Compute and display the optimal solution of the 01UKP solved using HiGHS as MIP solver.

setup the MIP solver to use:

In []:

compute the optimal solution:

In []:

display the results:

In []:

Technical note: use ``value( ukp[ :x1 ] )`` for referring a variable using the name of a model

Remark: in general, an MIP solver handles all decision variables as floating-point variables. The returned values may be approximations of integer values. Improve the

results accordingly by displaying the integer value.

Hints: see functions `convert()` and `round()` in the Julia documentation.

In []:

3. Compute the linear relaxation of the UKP using HiGHS as LP solver and display the solution.

Hints: see the type of variables of a JuMP model

In []:

Hints: see `relax_integrality()` in the JuMP documentation

In []:

4. Add an constraint and examine the optimal solution from different perspectives

Let's transform the optimization problem into a two-dimensional knapsack problem (01BKP) by adding the following constraint:

$$6x_1 + 2x_2 + 5x_3 + x_4 + 3x_5 \leq 10$$

1. Modify the model and solve it

Tasks: add this constraint to JuMP model and solve it with HiGHS. Print the optimal objective value and the selected variables.

Indication: let p , w_1 and w_2 be the vectors of coefficients for the profits and weights both constraints. Write the model in implicit form.

Hints: See the definition of a vector in the Julia documentation

Suggestion: name each constraint `cst1` and `cst2`, respectively.

In []:

2. Reading the solution

Task: Which items are selected? What is the remaining slack on each constraint?

Hints: see instruction `for` and `if`; see function `sum`

In []:

3. Simple sensitivity analysis

Task: If capacity of constraint 1 increases from 10 to 12, does the solution change?

Hints: see instruction `set_normalized_rhs`

In []:

Comments :

- Constraints must be named (e.g. `cst1` , `cst2`) when declared with `@constraint(model, cst1, ...)` so they can be referenced afterward.
- `set_normalized_rhs(constraint_ref, new_value)` changes the RHS in place. The model structure, variables, and objective stay untouched.

This is much faster than reconstructing the model, especially useful for parametric studies (e.g., looping over several capacity values).